
TAIAA 台灣AI應用推廣協會
2026第一次中級能力鑑定考前猜題&豪賭
主講人：胡小帥
中級
考前猜題&豪賭
時間：2026/05/22 20:00

KAGGLER'S CAPYBARA 指南: 起源與挑戰概覽

KAGGLER 的由來（Kaggle's Origin）



2010
Anthony Goldbloom

媒合企業難題與全球數據人才

被 Google 收購

主要題目類型（Competitions）

Featured（特色競賽）



商業價值最高
獎金優渥、難度最高

Research（研究競賽）



側重實驗性或學術性
推動特定領域技術邊界

Getting Started（入門競賽）



無獎金

專為新手設計
學習建模流程

Playground（遊樂場競賽）



難度介於入門與特色
題目有趣或具備啟發性

Recruitment（招聘競賽）



由公司贊助
篩選並招募頂尖人才

以 Kaggle 為題組來源的優缺點：

KAGGLER'S EXAM DESIGN GUIDE

優點 (Pros)

1 情境一致性高



考生不需反覆理解新的應用情境，考試節奏更順暢。

2 程式碼經過集體驗證



出題者引用的程式片段之正確性與穩定性極高，不用擔心語法有爭議。

3 實戰感強



Kaggle 數據包含噪音與缺失值，比課本上的「乾淨數據」更符合「場景實務」。

4 資源豐富



若考生想深究，考後有大量的開源資源可供學習，具有教育延續性。

缺點 (Cons)



1 答案易被背誦



經典賽題（如 Titanic）的處理流程幾乎是標準答案。對於有經驗的人來說，可能是在「背代碼」而非「考邏輯」。

2 過度依賴特定函式庫

pandas　scikit-learn

如果 IPAS 想要考核，Kaggle 題組可能會顯得過於「工具導向」。



Kaggle 題目強傾向使用 Pandas, Scikit-learn 要考核更底層的運算或原始。



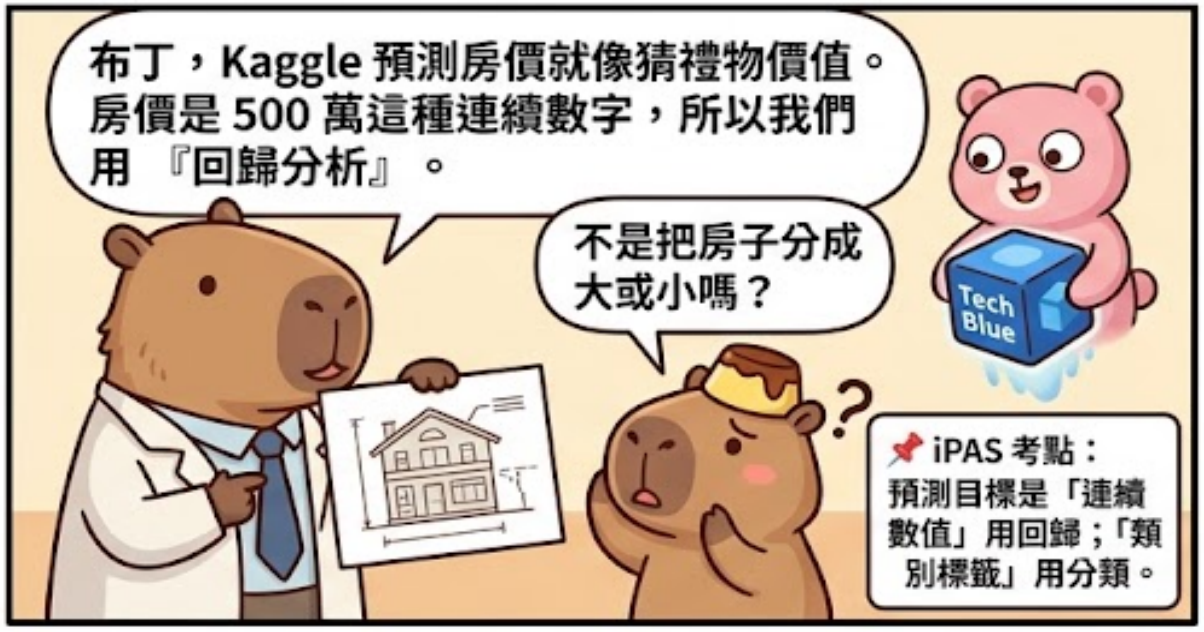
領域知識障礙

部分 Kaggle 競賽涉及專業領域（如基因序列），考生可能會因為「看不懂領域知識」而無法寫出 Python 代碼，導致鑑別度偏移。

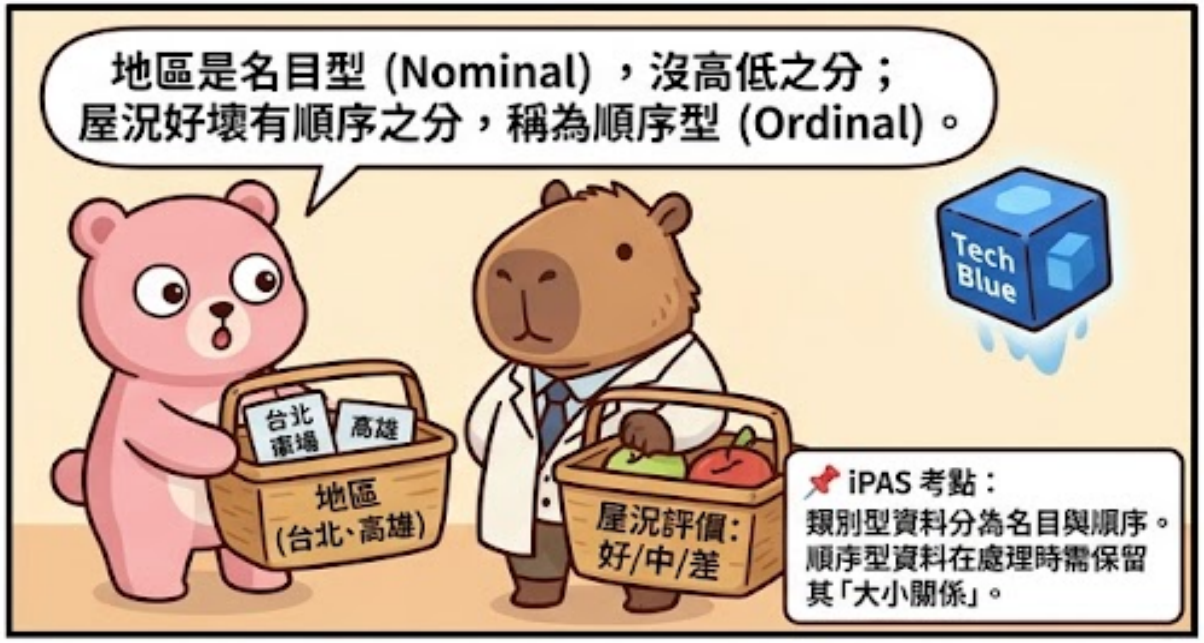
總結

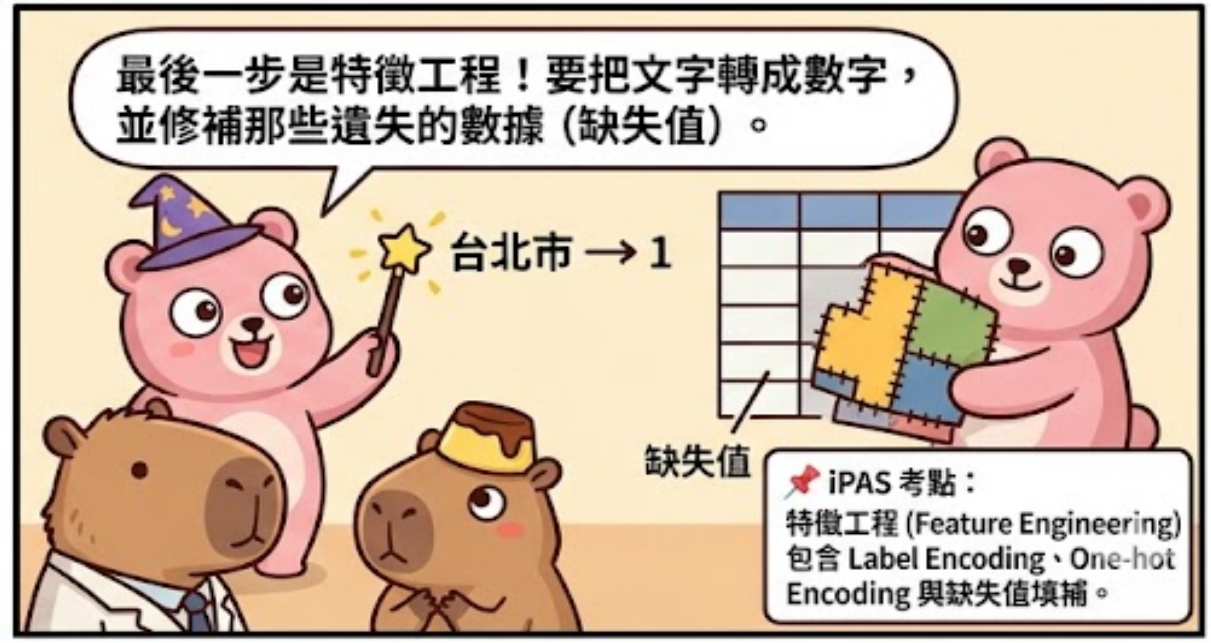


出題老師可能引用 Kaggle 的數據情境，但修改欄位名稱或邏輯細節」，既能保有「程式被驗證過」的優點，又能避免考生直接套用背下來的答案。


布丁，Kaggle 預測房價就像猜禮物價值。房價是 500 萬這種連續數字，所以我們用 『回歸分析』。
不是把房子分成大或小嗎？
Tech Blue
iPAS 考點：
預測目標是「連續數值」用回歸；「類別標籤」用分類。


房間數是 1，2，3 這種整數，這叫『離散型變數』。你不可能有 2.5 個房間，中間的數值是沒意義的。
房間數量：3
3
iPAS 考點：
離散型變數（Discrete）屬於定量資料，但數值間不連續。


地區是名目型 (Nominal)，沒高低之分；屋況好壞有順序之分，稱為順序型 (Ordinal)。
台北
高雄
地區
(台北、高雄)
屋況評價:
好/中/差
Tech Blue
iPAS 考點：
類別型資料分為名目與順序。順序型資料在處理時需保留其「大小關係」。


最後一步是特徵工程！要把文字轉成數字，並修補那些遺失的數據（缺失值）。
台北市 → 1
缺失值
iPAS 考點：
特徵工程 (Feature Engineering) 包含 Label Encoding、One-hot Encoding 與缺失值填補。

6.
在進行資料分析時，會遇到類別型（ Categorical ）與數值型 （ Numerical ） 資料格式。關於這兩種的處理，下列敘述何者不正確？
(A) One -Hot 編碼(One -Hot Encoding)會將類別變數轉換為多維二元向量，適用於無序（ Nominal ）類別資料，但在高基數(High Cardinality)特徵下可能造成維度爆炸問題 ；
(B) 標籤編碼(Label Encoding)會以整數表示不同類別，若應用於無序(Nominal)資料，可能導致模型誤將編碼值解讀為具數大小關係的特徵；
(C) 標準化(Standardization)透過將資料平移與縮放，使其均值為 0、標準差為 1，可在多數距離型演算法中改善收斂速度，並同時將值 範圍壓縮至 0 至 1 之間 ；
(D) 對連續變數進行分箱(Binning)可提升模型解釋性，但若分段方式未依據資料分佈特性設計，可能導致訊損失或邊界偏誤

17.
某電商公司欲利用顧客行為資料建立消費預測模型，其中「會員等級」欄位包含「一般、白金、黑卡」三種類別。若模型採用梯度提升樹（ Gradient Boosting Tree ）演算法，資料科學家在進行特徵編碼時應別注意下列何種情況？
(A) 應優先採用獨熱編碼(One -Hot Encoding)，以減少類別之間的相依性與**記憶體使用量**；
(B) 直接使用標籤編碼(Label Encoding)可能使模型誤判類別間存在順序關係，導致特徵重要性偏誤；
(C) 使用目標編碼(Target Encoding)會**自動消除過擬合(Overfitting)風險** ；
(D) 若類別數量較少，建議先使用主成分析(Principal Component Analysis, PCA)進行降維

1.Kaggle經典House Prices比賽，任務是根據房屋特徵來預測售價。

- 回歸分析:「房價」是連續性數值（例如500萬）屬於「回歸」而非「分類」。
  - 非連續型資料:
    - ✓ 離散型變數:整數，且中間沒有意義(如房間數量)
    - ✓ 類別型變數:名目型 (Nominal)和順序型 (Ordinal)
- 特徵工程: 文字轉換成數字，並處理缺失值。

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('house_data.csv')
df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].mean())
df = pd.get_dummies(df, columns=['MSZoning'])
X = df[['GrLivArea', 'OverallQual', 'LotFrontage']]
y = df['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
print(f"RMSE為: {rmse}")
```

- import pandas as pd
  - ✓ 負責資料清理與轉換(***Pandas:整理食材的工具***)
  - ✓ 將 CSV 檔案讀取為 DataFrame（類似 Excel 表格）進行處理缺失值、特徵篩選、資料轉換等
  - ✓ 考點：通常涉及 read_csv(), dropna(), get_dummies() (One-hot encoding)

- import numpy as np
  - ✓ ***Numpy:擅長數值計算、矩陣運算***
    - ➢ np.random(產生隨機數)、np.log(對數轉換)

Scikit-learn:烹飪器材。龐大Library， 透過 from ~ import~只加載需要的部分，節省記憶體與提升速度。

- from sklearn.model_selection import train_test_split
  - ✓ 將原始資料切分為「訓練集 (Train)」與「測試集 (Test)」
  - ✓ 參數test_size（通常設為 0.2)
  - ✓ random_state（固定隨機種子，確保實驗結果可重複）

- from sklearn.linear_model import **LinearRegression**
  - ✓ 載入線性迴歸演算法。

- from sklearn.metrics import **mean_squared_error**
  - ✓ 模型效能評估，計算均方誤差 (MSE) 迴歸分析最常用的評估指標
  - ✓ 匯入 MAE 與 R2 評估函數:*from sklearn.metrics import* ***mean_absolute_error, r2_score***

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('house_data.csv')
df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].mean())
df = pd.get_dummies(df, columns=['MSZoning'])
X = df[['GrLivArea', 'OverallQual', 'LotFrontage']] y = df['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
print(f"模型的均方根誤差 (RMSE) 為: {rmse}")
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('house_data.csv')
# 1. 讀取數據 (數據集名稱為 house_data.csv)

df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].mean())
# 2. 特徵工程：計算LotFrontage(房屋連接街道的直線距離) 欄位的平均值，並用來填補缺失值（NaN）
✓ 中位數:median() : 純量，只有一個結果，直接填補。
✓ 眾數mode()[0] :可能有多個結果，從「眾數清單」中抽出第一個值。

df = pd.get_dummies(df, columns=['MSZoning'])
# 3. 特徵工程：將columns MSZoning(土地分區性質) 欄位之類別轉為數值 (One-Hot Encoding)
```

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.get_dummies(df, columns=['MSZoning'])
# 3. 特徵工程：將columns MSZoning(土地分區性質)欄位之類別轉為數值 (One-Hot Encoding)
```

✓ 「有順序關係」特徵（如：良好=3, 普通=2, 差=1），**改用標籤編碼 (Label Encoding)**

✓ **特徵縮放 (StandardScaler)： 將數據標準化（均值 0，標準差 1）**

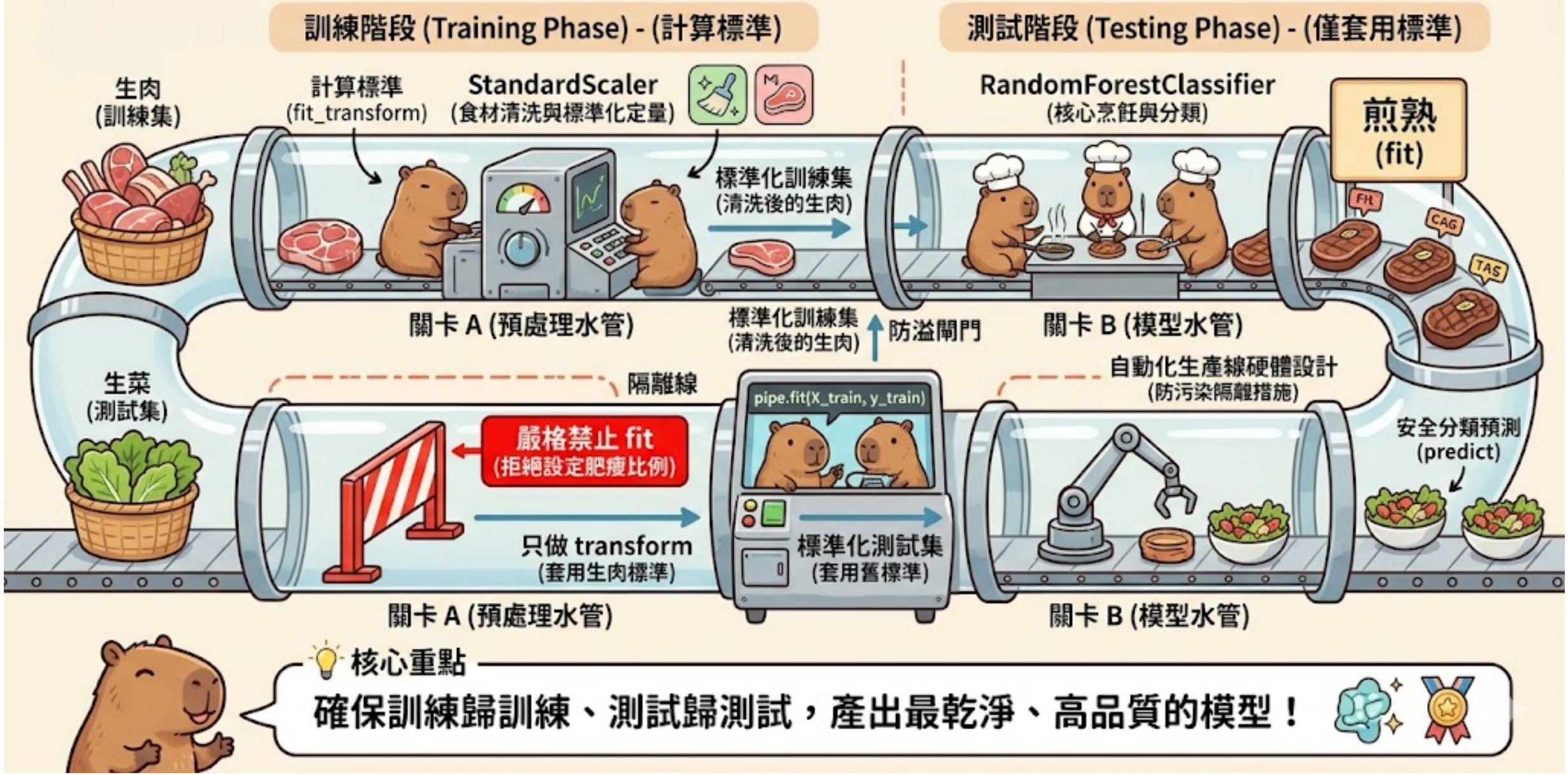
```
from sklearn.preprocessing import LabelEncoder, StandardScaler
le = LabelEncoder()
df['MSZoning'] = le.fit_transform(df['MSZoning'].astype(str))
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

- ✓ **fit：計算出訓練集裡特徵欄位的「平均值」與「標準差」。**
- ✓ **transform：利用平均值和標準差，把訓練集的數值轉換成標準化的數值**

```
X_test_scaled = scaler.transform(X_test)
```

- ✓ **測試集只能使用「從訓練集學到的平均值和標準差」來進行縮放， 如果測試集也拿去fit，就會造成資料洩漏(Data Leakage)，這是嚴重的錯誤。**

Pipeline (自動化與無菌隔離的模型生產線)



🚫 不要讓 AI 偷看未來的劇本！

✅ 時間序列資料切分對策 (Look-ahead bias vs. TimeSeriesSplit)

以歷史考試為比喻，理解錯誤對策 (KFold) 與正確對策 (TimeSeriesSplit)

❌ 傳統隨機切分 (分 - KFold)

❌ 錯誤：傳統隨機切分 (KFold) ｜ 致命盲點：時間旅行型資料洩漏 (Look-ahead bias)



結果：考試拿滿分 (準確度虛胖)
真實世界瞬間放空，慘賠！(無未來劇本可看)

✅ 正確對策 - TimeSeriesSplit

✅ 正確：時序切分法 (TimeSeriesSplit) ｜ 科學解法：滾動擴展式切分 (Expanding Window)



第 1 次模擬 (Fold 1)		第 2 次模擬 (Fold 2)		第 3 次模擬 (Fold 3)	
Known (訓練)	預測 (測試)	Known (訓練)	預測 (測試)	Known (訓練)	預測 (測試)
1900 ~ 1920	1921 ~ 1930	1900 ~ 1930	1931 ~ 1940	1900 ~ 1940	1941 ~ 1950

結果：順著時間軸延伸訓練集，
學會基於『既有歷史』推估『未知明天』的真功夫！

💡 核心重點

「過去的經驗只能用來預測未來，
絕對不能用未來的記憶來欺騙過去。」

透過滾雪球方式順時序訓練，AI 才能真正預測未知明天。

```
X = df[['GrLivArea', 'OverallQual', 'LotFrontage']]
y = df['SalePrice']
```
4. 定義關鍵特徵 X 與目標物 y

- GrLivArea (Above Grade Living Area)：地面上的坪數。
- OverallQual (Overall Quality)：房屋的整體材料與完工品質（通常是 1-10 分）。
- LotFrontage：已補完平均值的街道長度。

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```
5. 切割資料集 (80% 訓練, 20% 測試)

✓ random_state=42:追求實驗公平，每次執行，訓練集和測試集的內容完全相同。

✓ 經典科幻小說《銀河便車指南》中提到：「42 是生命、宇宙以及任何事情的終極答案。」從此 42 就成了資料科學界最常見的默認數字。

✓ K 折交叉驗證 (K-Fold Cross-Validation)： 將資料分成 **K** 份，輪流當測試集，最後取平均值。

- from sklearn.model_selection import cross_val_score
- scores = cross_val_score(model, X, y, ***cv=5***, scoring='neg_root_mean_squared_error')

```
model = LinearRegression()
model.fit(X_train, y_train)
```
6. 訓練回歸模型

✓ 增加「懲罰項」來防止過擬合>改用 Lasso 回歸

- from sklearn.linear_model import Lasso
- model = Lasso(alpha=1.0)
- model.fit(X_train_scaled, y_train)

```
predictions = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
print(f"RMSE為: {rmse}")

# 7. 預測與評估
```

✓ 平均絕對誤差(MAE)：直觀好懂。
✓ R^2：介於 0 到 1 之間，數值越接近 1，代表模型對數據的解釋力越強（0.85 代表模型能解釋 85% 的房價波動）

- mae = mean_absolute_error(y_test, predictions)
- r2 = r2_score(y_test, predictions)
- print(f"平均絕對誤差 (MAE): {mae:.2f}")
- print(f"決定係數 (R-squared): {r2:.4f}")
  - .2f 保留到小數點後2位(常用於 MAE、金額、百分比)
  - .4f 保留到小數點後4位。(常用於 R^2、精確度、機率值)
  - .0f 四捨五入到整數
  - .2f 加上千分位逗號
  - .2% 轉成百分比

理解原理後 讓我們再看一次程式碼，
原本冷漠的指令，是不是瞬間變成熟悉的好友了呢?

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

df = pd.read_csv('house_data.csv')
df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].mean())
df = pd.get_dummies(df, columns=['MSZoning'])
X = df[['GrLivArea', 'OverallQual', 'LotFrontage']]
y = df['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
print(f"RMSE為: {rmse}")
```


布丁，我們要在Kaggle挑戰經典的Titanic題目！預測乘客能生還嗎？
Kaggle經典
kaggle
0死亡
1生還
乘客能不能活下來？好沉重哦…
Tech Blue
iPAS考試關鍵：分類 vs. 回歸
分類分析：預測離散值（0死亡、1生還）
回歸分析：預測連續值（例如：年齡、票價）。Titanic是分類題！
沒錯，生還（0/1）是非連續的資料。數據中有很多變數，比如性別。
男
女
性別
非連續
性別只有Male和Female，沒有中間值吧？
Tech Blue
iPAS變數類型：名目型 (Nominal)
類別型資料的一種。
名目型變數：性別（男/女）、地區（台北/臺雄），資料之間沒有順序或大小關係。
對！這叫『名目型』。另一種是『順序型』，比如艙等（Pclass）。
1st
2nd
>
3rd
>
3rd
就像1st > 2nd > 3rd？數值大小有邏輯意義！
Tech Blue
iPAS變數類型：順序型 (Ordinal)
類別型資料的一種。
順序型變數：艙等、滿意度 (好/普通/溫)，資料之間
具有明確的順序關係。iPAS常考！
布丁，機器只懂數字，要要把文字轉換成數字，並填補缺失值（Missing Value）！
缺失值
10
1
1
看我的『特徵工程魔法』！
Male: 10
1
Tech Blue
iPAS關鍵：類別編碼 (One-hot vs. Label)
用 get_dummies 進行 One-hot Encoding：
- 適用無順序變數 (性別)，將文字轉為數字。
用 LabelEncoder：適用具順序性類別 (艙等)。

2.Kaggle經典Titanic 比賽：任務是根據乘客特徵（年齡、性別、艙等）來預測其是否生還 。

- 分類分析 (Classification)：0（死亡）或 1（生還） 的離散數值 。

非連續型資料:

- ✓ 名目型變數：如性別 (Male/Female)，中間沒有順序意義 。
- ✓ 順序型變數：如艙等 (1st > 2nd > 3rd)，數值大小具備邏輯意義 。

- 特徵工程: 文字轉換成數字，並處理缺失值。
  - ✓ 使用 get_dummies 進行 One-hot Encoding 將文字轉為數字 。
  - ✓ 使用 LabelEncoder 處理具順序性的類別 。

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('titanic.csv')
df = pd.get_dummies(df, columns=['Sex'], drop_first=True)
X = df[['Pclass', 'Age', 'Sex_male']]
y = df['Survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
cm = confusion_matrix(y_test, predictions)
print("混淆矩陣：")
print(cm)
print("\n分類報告：")
print(classification_report(y_test, predictions))
```

- import pandas as pd

✓ 負責資料清理與轉換(***Pandas:整理食材的工具***)

✓ 將 CSV 檔案讀取為 DataFrame（類似 Excel 表格）進行處理缺失值、特徵篩選、資料轉換等

✓ 考點：通常涉及 read_csv(), dropna(), get_dummies() (One-hot encoding)

Scikit-learn:烹飪器材。龐大Library， 透過 from ~ import~只加載需要的部分，節省記憶體與提升速度。

- from sklearn.model_selection import train_test_split

✓ 將原始資料切分為「訓練集 (Train)」與「測試集 (Test)」

✓ 參數test_size（通常設為 0.2)

✓ random_state（固定隨機種子，確保實驗結果可重複）

- from sklearn.linear_model import ***LogisticRegression***

✓ 載入邏輯回歸演算法。

  - Lasso 回歸 (L1 Regularization):將不重要的特徵係數直接強迫歸零
  - Ridge 回歸 (L2 Regularization):將特徵係數逼近於0，但不會等於 0

- from sklearn.metrics import ***confusion_matrix, classification_report***

✓ 模型效能評估，混淆矩陣為評估指標

增加「懲罰項」來防止過擬合

ADD 'PENALTY TERM' TO PREVENT OVERFITTING

LASSO 回歸 (L1 正規化)

將不重要的特徵係數直接強迫歸零




```
from sklearn.linear_model import Lasso;
model = Lasso(alpha=1.0);
model.fit(X_train_scaled, y_train)
```

RIDGE 回歸 (L2 正規化)

將特徵係數逼近於 0，但不會等於 0




```
from sklearn.linear_model import Ridge;
model = Ridge(alpha=1.0);
model.fit(X_train_scaled, y_train)
```

核心參數 `alpha=1.0 (處罰力道旋鈕)



alpha = 0
(過擬合 Overfitting)

1.0

alpha = 100
(欠擬合 Underfitting)



LASSO 是無情的「剪刀」，不重要就直接砍掉 (0)；RIDGE 是溫和的「緊箍咒」，讓大家都變小，但一個都不會少 (→0)。

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('titanic.csv')
# 1. 讀取數據 (數據集名稱為 titanic.csv)
df = pd.get_dummies(df, columns=['Sex'], drop_first=True)
# 使用 One-hot Encoding 處理性別
```

✓ drop_first=True，Pandas自動刪除第一個類別欄位。

設定方式	轉換後的欄位	說明
drop_first=False (預設)	Sex_Female, Sex_Male	兩個欄位互為鏡像，存在共線性。
drop_first=True	Sex_Male	最精簡。以「女性」為基準，1 代表男，0 代表女。

```
X = df[['Pclass', 'Age', 'Sex_male']]
y = df['Survived']
# 2. 定義特徵 X 與目標 y
X=艙等、年齡、性別
 Y= 0 ( 死亡 ) 或 1 ( 生還 )
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```
3. 切割資料集 (80% 訓練, 20% 測試)

✓ *random_state=42:追求實驗公平，每次執行，訓練集和測試集的內容完全相同。*

```
model = LogisticRegression()
model.fit(X_train, y_train)
```
4. 訓練分類模型: 邏輯回歸預設使用 Sigmoid 激活函數

透過 MLPClassifier 可以切換不同的激活函數 。

```
from sklearn.neural_network import MLPClassifier
```

- *使用 ReLU (目前神經網路最常用的激活函數)*
  - ✓ `model_relu = MLPClassifier(activation='relu', hidden_layer_sizes=(64, 32), random_state=42)`
  - ✓ `model_relu.fit(X_train, y_train)`

- *使用 Tanh (雙曲正切函數，輸出介於 -1 到 1)*
  - ✓ `model_tanh = MLPClassifier(activation='tanh', hidden_layer_sizes=(64, 32), random_state=42)`
  - ✓ `model_tanh.fit(X_train, y_train)`

- *使用 Softmax*
  - ✓ *當y包含3個以上的類別時，MLPClassifier 輸出層會自動採用Softmax*

摘要：訓練分類模型 —— 激活函數的四大金剛

白話比喻：神經元的「決策過濾器」，負責非線性轉換，決定資訊傳遞。

Sigmoid

二分法裁判

非黑即白：將數值壓縮為機率





垃圾郵件分類

輸出值域：0 到 1

Tanh (雙曲正切)

正負評量尺

有情緒：零中心化，收斂較快





帶有正負的特徵

輸出值域：-1 到 1

ReLU (修正線性單元)

現實主義者

沒用就滾：負數歸零，計算極快





深度網絡預設首選

(目前最常用)

輸出值域：0 到 +∞

Softmax

分餅財務長

機率分餅：總和必定 100%





多類別輸出層

所有類別機率相加 = 1

Scikit-Learn 程式碼範例卡

```python
from sklearn.neural_network import MLPClassifier
# 使用 ReLU
model_relu = MLPClassifier(activation='relu', hidden_layer_sizes=(64, 32))
# 使用 Tanh
model_tanh = MLPClassifier(activation='tanh', hidden_layer_sizes=(64, 32))
```

金句：Sigmoid 辦二選一，Tanh 帶負好收斂；ReLU 負數歸零跑最快，Softmax 多選一分機率。

```
predictions = model.predict(X_test)
cm = confusion_matrix(y_test, predictions)
print("混淆矩陣：")
print(cm)
print("\n分類報告：")
print(classification_report(y_test, predictions))
# 5. 預測與評估

    acc = accuracy_score(y_test, predictions) # 準確率
    prec = precision_score(y_test, predictions) # 精確率
    rec = recall_score(y_test, predictions) # 召回率
    f1 = f1_score(y_test, predictions) # F1-Score
```

再看我(程式碼)一眼，現在有幾分熟了?

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

df = pd.read_csv('titanic.csv')
df = pd.get_dummies(df, columns=['Sex'], drop_first=True)
X = df[['Pclass', 'Age', 'Sex_male']]
y = df['Survived']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
cm = confusion_matrix(y_test, predictions)
print("混淆矩陣：")
print(cm)
print("\n分類報告：")
print(classification_report(y_test, predictions))
```


布丁，這就是 Kaggle 最經典的鳶尾花（Iris）數據集。我們的任務是看花萼和花瓣的特徵，來預測它是哪種品種喔！
iPAS 核心考點：
「品種」是離散的類別標籤（如山鳶尾、變色鳶尾），預測目標屬於「分類分析」！
那這些品種有分第一名、第二名，或者誰比較大嗎？
完全沒有！這叫名目型變數，它們之間沒有大小、順序或權重之分。
=
因為 AI 看不懂文字，所以在特徵工程中，我們要用 Label Encoding 把品種名稱換成 0, 1, 2 這些數字編碼！
「山鳶尾」「變色鳶尾」
「維吉尼亞」
0,1,2
哇！
避坑指南來囉！
大家這裡最容易踩雷！
Tech Blue
iPAS 避坑指南：
為什麼是 Label Encoding 而不是 One-Hot 呢？因為「品種」是預測目標（Y），主流分類模型底層可以直接吃 0, 1, 2。用 Label 才能保持維度簡潔、不浪費記憶體喔！

3.Kaggle經典Iris（鳶尾花）比賽，任務是根據花萼與花瓣特徵來預測鳶尾花的品種。

- 分類分析:「品種」是離散的類別標籤（例如山鳶尾、變色鳶尾、維吉尼亞鳶尾）屬於「分類」而非「回歸」。
- 非連續型資料: 類別型變數: 名目型 (Nominal)，三個品種之間並無大小、順序或權重之分。
- 特徵工程: 將品種的文字標籤轉換成數字編碼（Label Encoding，如 0, 1, 2），並檢視資料分佈與完備性。

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# 載入內建鳶尾花資料集
iris = sns.load_dataset("iris")

# 查看前五筆資料
# 特徵：sepal_length (花萼長), sepal_width (花萼寬), petal_length (花瓣長), petal_width (花瓣寬)
# 目標：species (品種：setosa, versicolor, virginica)
print(iris.head())print(f"RMSE為: {rmse}")
```

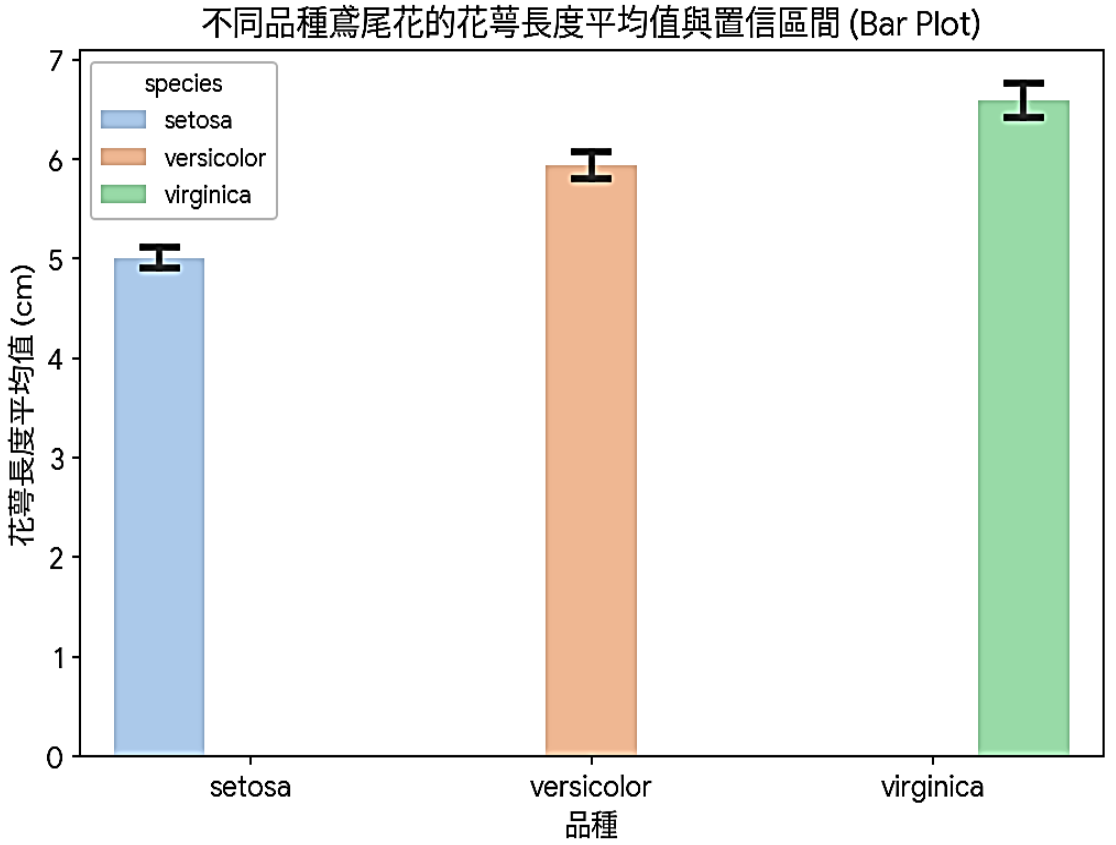
- import pandas as pd
  - ✓ 負責資料清理與轉換(***Pandas:整理食材的工具***)
  - ✓ 將 CSV 檔案讀取為 DataFrame（類似 Excel 表格）進行處理缺失值、特徵篩選、資料轉換等
  - ✓ 考點：通常涉及 read_csv(), dropna(), get_dummies() (One-hot encoding)

- import numpy as np
  - ✓ ***Numpy:擅長數值計算、矩陣運算***
    - ➢ np.random(產生隨機數)、np.log(對數轉換)

- import matplotlib.pyplot as plt
  - ✓ ***Matplotlib***：資料視覺化的基礎工具，可自訂圖表的所有細節（如標題、座標軸與字體設定）。
  - ✓ 考點：通常涉及 plt.figure() (設定畫布), plt.title(), plt.show() (渲染並顯示圖表)。

- import seaborn as sns
  - ✓ **Seaborn**：高級統計圖表專家，擅長用極簡的程式碼繪製出美觀且具備統計意義的複雜圖表。
  - ✓ 考點：通常涉及 sns.set_theme() (切換風格), sns.histplot() (分佈圖), sns.heatmap() (熱力圖)。

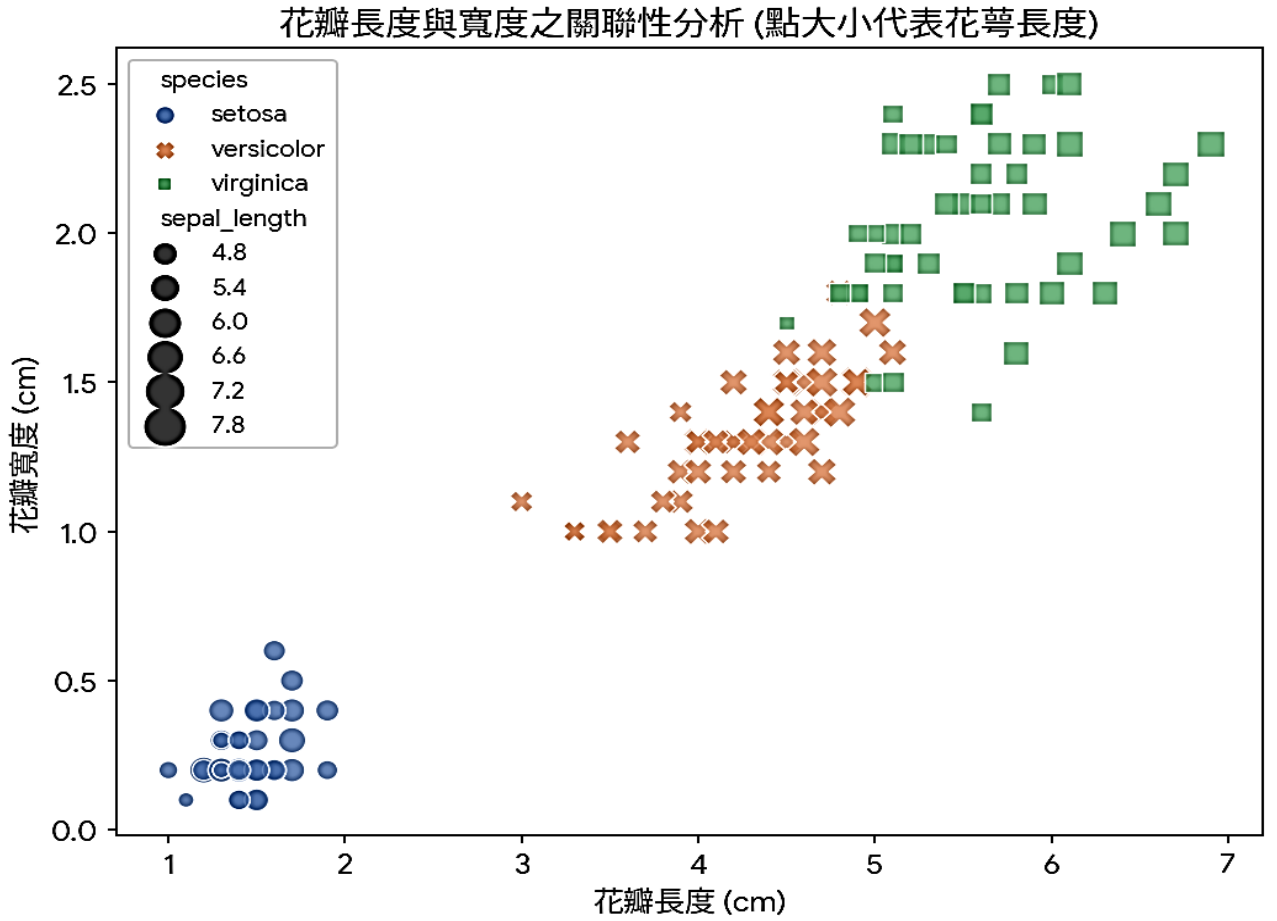
```
plt.figure(figsize=(8, 5))
sns.barplot(
    data=iris,
    x="species",
    y="sepal_length",
    hue="species",  # 依品種分色
    estimator=np.mean,  # 統計量：平均值
                        (或改為 np.median)
    errorbar=("ci", 95),  # 顯示 95% 置信區間的誤差棒
    capsize=0.1,  # 誤差棒頂部橫線的寬度
    palette="pastel",
)
```




```
plt.title("不同品種鳶尾花的花萼長度平均值與置信區間 (Bar Plot)", fontsize=14)
plt.xlabel("品種", fontsize=12)
plt.ylabel("花萼長度平均值 (cm)", fontsize=12)
plt.show()
```

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=iris,
    x="petal_length",
    y="petal_width",
    hue="species", # 顏色代表品種
    style="species", # 形狀代表品種
    size="sepal_length", # 點的大小隨花萼長度變化
    sizes=(40, 240), # 控制點的最大與最小尺寸範圍
    alpha=0.85,
    palette="deep",
)

plt.title("花瓣長度與寬度之關聯性分析 (點大小代表花萼長度)",
fontsize=14)
plt.xlabel("花瓣長度 (cm)", fontsize=12)
plt.ylabel("花瓣寬度 (cm)", fontsize=12)
plt.show()
```

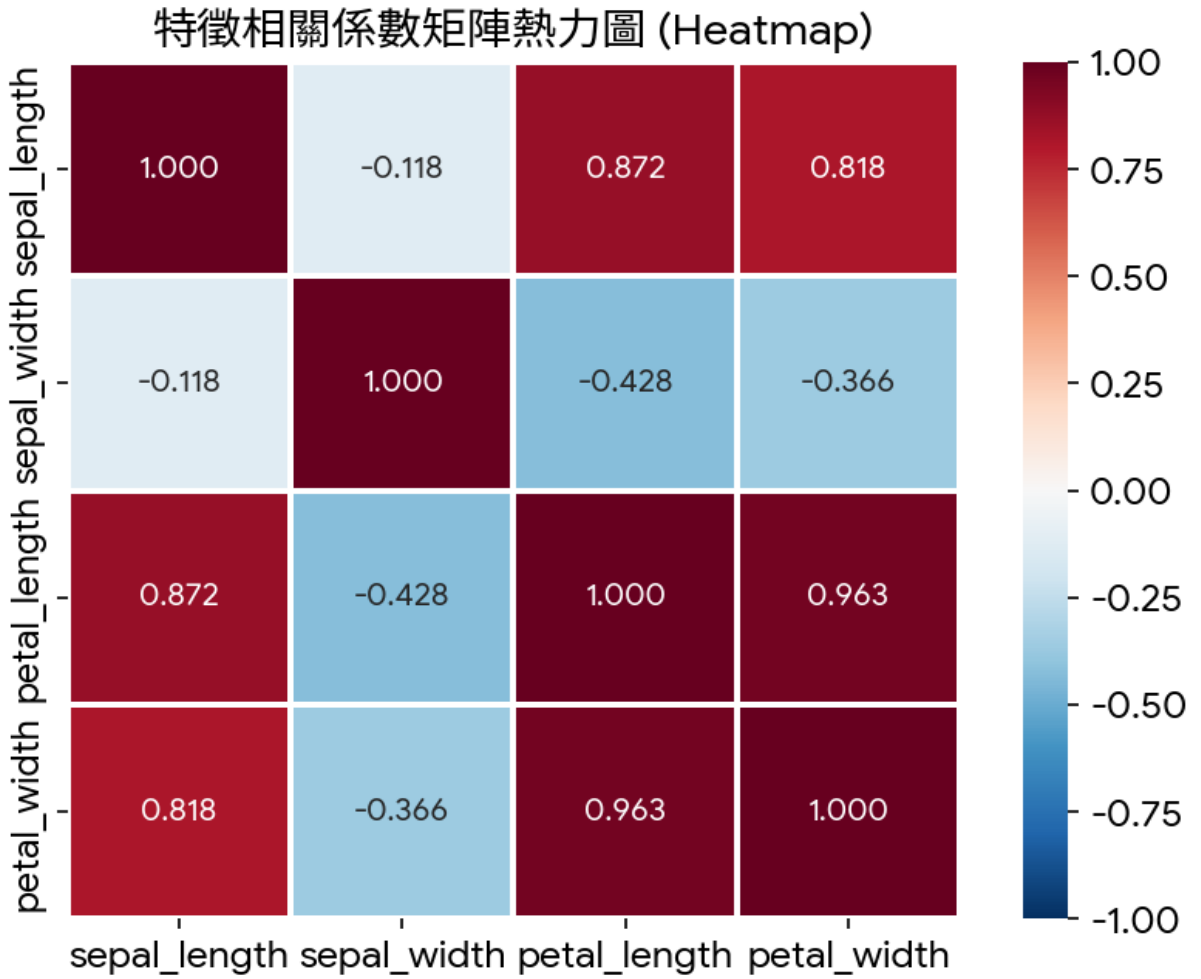


```
# 計算相關係數矩陣
corr_matrix = iris.drop(columns=["species"]).corr()
```

相關係數（Pearson correlation）只能計算數值型欄位，
先用 .drop() 把文字型態的品種欄位（species）排除，
再用 .corr() 計算其餘四個特徵兩兩之間的相關性。

```
plt.figure(figsize=(7, 5))
sns.heatmap(
    corr_matrix,
    annot=True, # 顯示相關係數數值
    fmt=".3f", # 數值格式保留三位小數
    cmap="RdBu_r", # 紅藍發散色帶 (Red-Blue reversed)
    vmin=-1,
    vmax=1, # 固定上下界以利客觀評估
    linewidths=1, # 各格子間的間隔線寬
    linecolor="white", # 間隔線顏色
    square=True, # 強制讓格子維持正方形
)

plt.title("特徵相關係數矩陣熱力圖 (Heatmap)", fontsize=14)
plt.show()
```



```
# 統計各品種的數量
species_counts = iris["species"].value_counts()
```

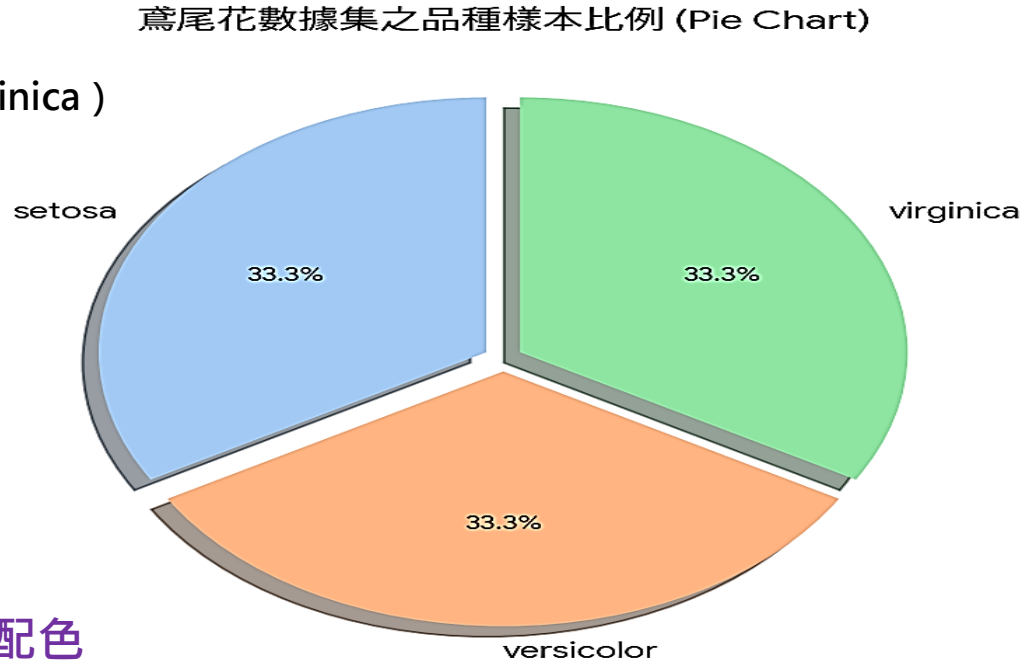
計算species欄位中每個不重複品種出現的次數，三個品種（setosa, versicolor, virginica）的數量各是 50 筆。

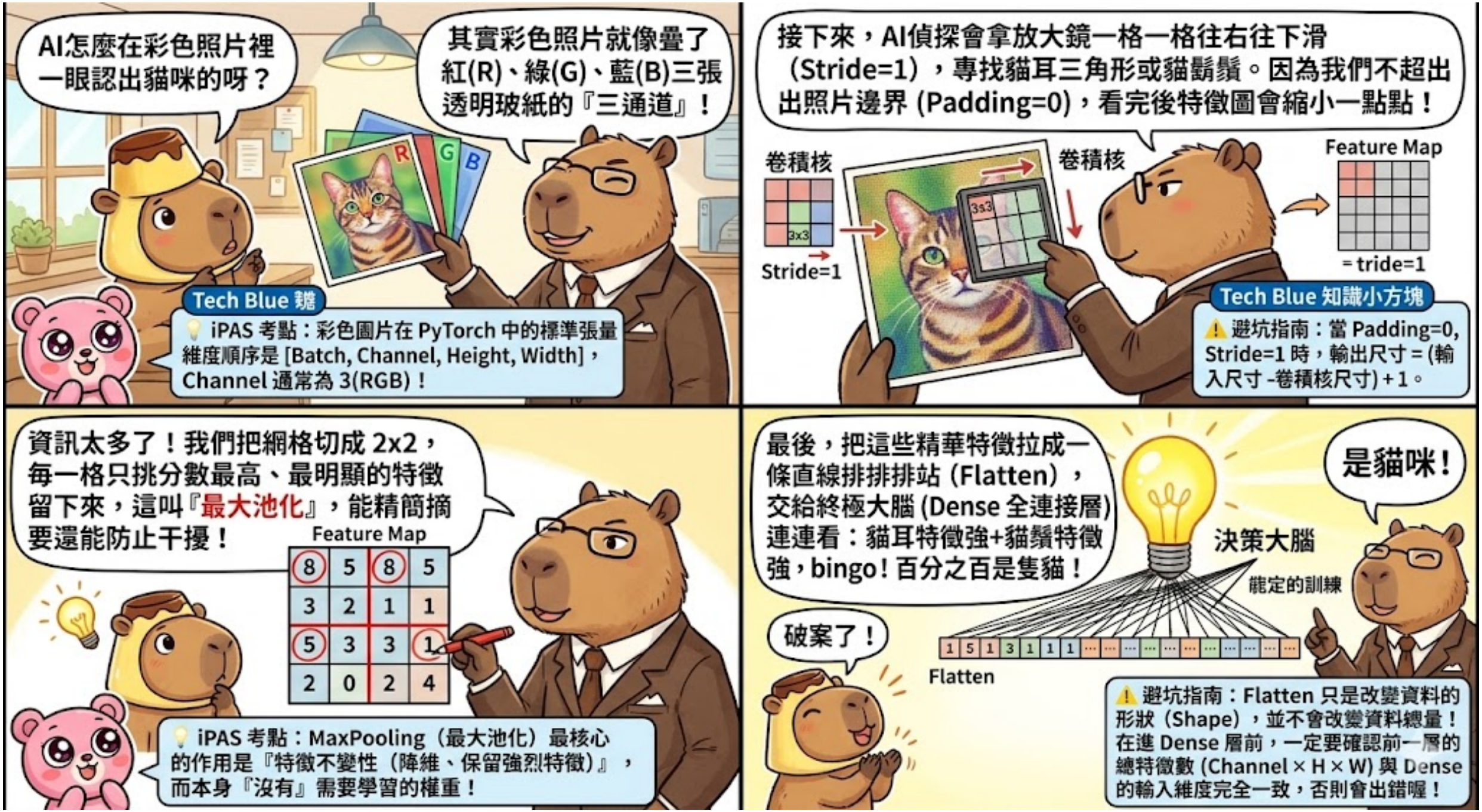
```
plt.figure(figsize=(6, 6))
plt.pie(
```

呼叫 Matplotlib 的繪圖函式

```
    species_counts,
    labels=species_counts.index,
    autopct="%1.1f%%",  # 顯示百分比格式
    startangle=90,  # 起始旋轉角度
    colors=sns.color_palette("pastel")[0:3],  # 沿用 Seaborn 的柔和配色
    explode=(0.05, 0.05, 0.05),  # 將各區塊稍微分離，增加視覺層次感
    shadow=True,  # 啟用立體陰影
)

plt.title("鳶尾花數據集之品種樣本比例 (Pie Chart)", fontsize=14)
plt.show()
```




AI怎麼在彩色照片裡一眼認出貓咪的呀？
其實彩色照片就像疊了紅(R)、綠(G)、藍(B)三張透明玻紙的『三通道』！
R
G
B
Tech Blue 豬
iPAS 考點：彩色圖片在 PyTorch 中的標準張量維度順序是 [Batch, Channel, Height, Width]，Channel 通常為 3(RGB)！
接下來，AI偵探會拿放大鏡一格一格往右往下滑（Stride=1），專找貓耳三角形或貓鬍鬚。因為我們不超出出照片邊界 (Padding=0)，看完後特徵圖會縮小一點點！
卷積核
3x3
Stride=1
卷積核
3x3
Feature Map
= tride=1
Tech Blue 知識小方塊
避坑指南：當 Padding=0, Stride=1 時，輸出尺寸 = (輸入尺寸 -卷積核尺寸) + 1。
資訊太多了！我們把網格切成 2x2，每一格只挑分數最高、最明顯的特徵留下來，這叫『最大池化』，能精簡摘要還能防止干擾！
Feature Map
8 5 8 5
3 2 1 1
5 3 3 1
2 0 2 4
iPAS 考點：MaxPooling（最大池化）最核心的作用是『特徵不變性（降維、保留強烈特徵）』，而本身『沒有』需要學習的權重！
最後，把這些精華特徵拉成一條直線排排排站（Flatten），交給終極大腦 (Dense 全連接層) 連連看：貓耳特徵強+貓鬚特徵強，bingo! 百分之百是隻貓！
決策大腦
是貓咪！
破案了！
1 5 1 3 1 1 1 ··· ··· ··· ··· ··· ··· ··· ··· ··· ···
Flatten
避坑指南：Flatten 只是改變資料的形狀（Shape），並不會改變資料總量！在進 Dense 層前，一定要確認前一層的總特徵數 (Channel × H × W) 與 Dense 的輸入維度完全一致，否則會出錯喔！

水豚的影像分類冒險 (Capybara's Image Classifics) - 神經網絡步驟詳解

1 原始圖片進場

輸入 [32張 (32 images), 3通道 (RGB channels), 32x32像素 (pixels)]

輸入 = [32, 3, 32, 32]



2 卷積層 (Convolution Layer)

進行 16 個特徵偵測

[32, 16, 30, 30]

計算過程

- 圖片尺寸 (Input Dim): 32
- 濾波器 (Filter): 3×3
- 填充 (Padding): 0
- 步長 (Stride): 1

$$\left(\frac{32-3+2\times0}{1}\right)+1=30$$

輸出尺寸 = [32, 16, 30, 30]



3 池化層

特徵濃縮

計算過程

- 圖片尺寸
- 池化步長



(30 / 2) = 15



H，W 消失

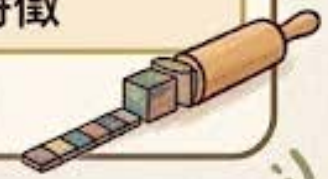
4 攤平層 (Flattening Layer)

將 3D 特徵圖展平成 1D 特徵向量

計算過程

16 通道×15 高度×15 寬度 = 3600 個特徵

輸出尺寸 = [32, 3600]



5 全連接層 (Dense/FC Layer)

輸出為 10 個數字的分類

計算過程

- 輸入特徵數: 3600
- 輸出類別數: 10

輸出尺寸 = [32, 10]



Key: Tensor 計算 Illustration

https://dashboard.blooket.com/my-sets



摘要：Box-Cox 轉換 —— 數據的「變形金剛」

(白話比喻)：它是數據的「魔法更衣鏡」：自動調校黃金比例，整形回標準常態名模，線性迴歸完美預測。

核心機制：λ (Lambda) 自動優化旋鈕對比表

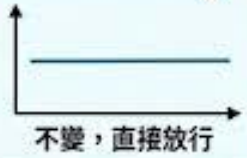
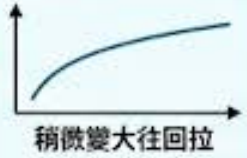
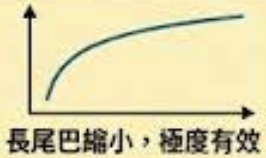
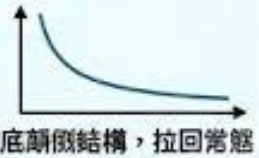
λ (Lambda) 旋鈕
自動尋找最佳參數



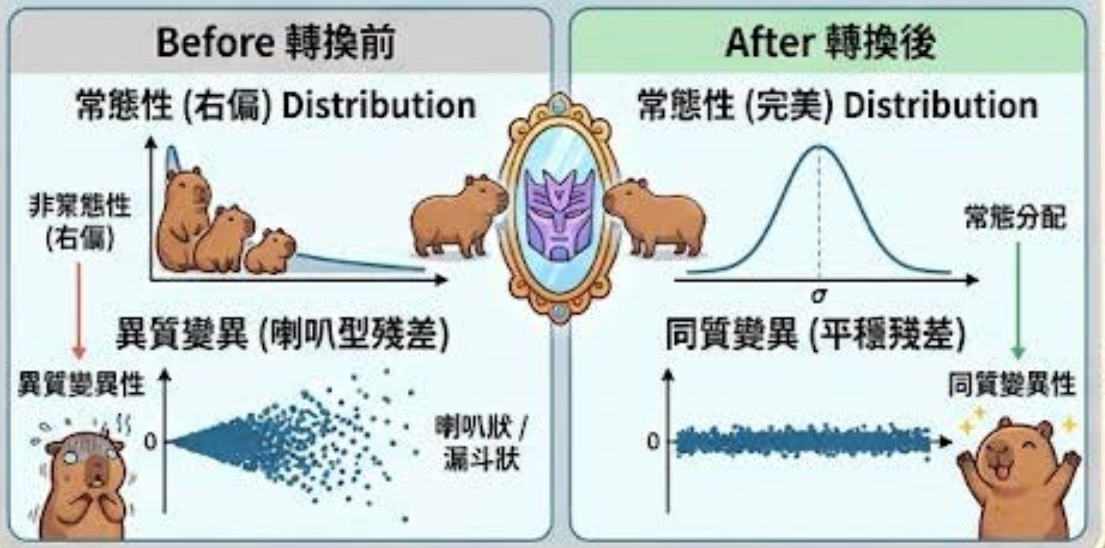
λ (Lambda) 旋鈕
自動尋找最佳參數



⚠ 大前提：原始數值必須全部大於 0 ($x > 0$)

λ = 1.0 維持原樣 $y = x$		
λ = 0.5 開根號 $y = \sqrt{x}$		
λ = 0.0 嚴重右偏 / 自然對數 $y = \ln(x)$		
λ = -1.0 倒數 $(y = 1/x)$		

資料圖表：轉型前後 (Before vs After) 視覺化設計指南



數據工程師的 Scikit-Learn / SciPy 程式碼字卡

```
from scipy import stats
# 原始數值 X
transformed_data, best_lambda = stats.boxcox(raw_data)
print(f"🧙 最佳 λ 旋鈕參數: {best_lambda:.4f}")
```

金句：數據怎麼歪、殘差怎麼炸都不怕！Box-Cox 的 λ 旋鈕一轉，極端數據秒變標準常態名模，線性迴歸完美過關！